PYTHON ACTIVITY-BASED LEARNING

CONSOLIDATED CASE STUDY REPORT

Course: Python Programming

Student Details

Name: Modi Yasrija

Roll Number: 26000952

Branch: CSM

Section: 2

Institution: Dr. RVR NRI Institute of Technology

Academic Year: 2026–2027

Table of Contents

1. Case Study 12: Attendance Recovery Computation
2. Case Study 14: Warehouse Inventory Reconciliation
3. Case Study 17: Student Performance Trend Analysis
4. Case Study 28: Repeated Transaction Detection
5. Case Study 31: String-Based Examination Answer Evaluation
6. Case Study 35: Text Compression Using Character Runs
7. Case Study 42: Student Marks Matrix Analysis
8. Case Study 49: Tuple-Based Product Catalogue Audit
9. Case Study 55: Tuple-Based Transaction Audit
10. Case Study 66: Student Academic Risk Classification

Case Study 12: Attendance Recovery Computation

Problem Statement

Students must maintain the minimum attendance percentage required by their college. This program calculates the student's current attendance percentage based on the total number of classes conducted and the number of classes attended.

If the student's attendance is below the required 75%, the program calculates the minimum number of additional consecutive classes the student must attend without missing any classes to reach the required percentage. If the attendance requirement is already satisfied, the program displays a suitable message.

The program also validates the input values to avoid invalid attendance data.

Objectives

1. To calculate the student's current attendance percentage.
2. To check whether the student satisfies the minimum attendance requirement of 75%.
3. To calculate the minimum number of additional consecutive classes needed to reach 75% attendance.
4. To validate the input values and display appropriate messages for invalid data.
5. To practise Python variables, arithmetic operators, conditional statements, and loops.

Assumptions

1. The minimum required attendance percentage is 75%.
2. Total classes conducted must be greater than zero.
3. Classes attended must be between zero and the total number of classes conducted.
4. The student attends every additional consecutive class without missing any.
5. The required number of additional classes is calculated using the current attendance figures.

Algorithm

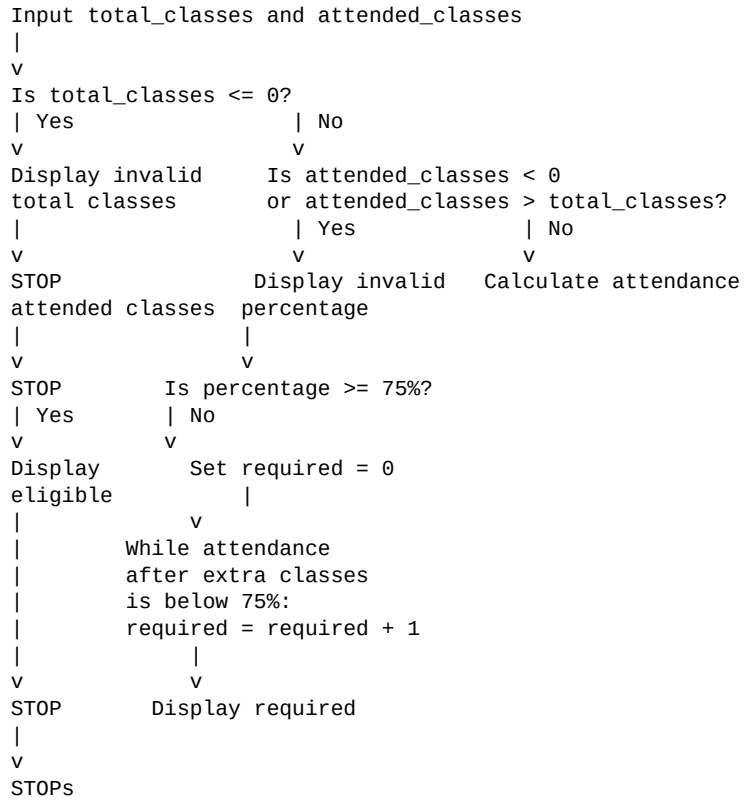
```
Algorithm: Attendance Recovery Computation

1. Start.
2. Input the total number of classes conducted.
3. Input the number of classes attended.
4. Check whether the total classes are greater than zero.
5. If not, display "Invalid total classes" and stop.
6. Check whether attended classes are between zero and total classes.
7. If not, display "Invalid number of classes attended" and stop.
8. Calculate attendance percentage using:
   Attendance Percentage = (Attended Classes / Total Classes) × 100.
9. Display the current attendance percentage.
10. If attendance is at least 75%, display "Attendance requirement satisfied".
11. Otherwise, initialize the number of additional classes to zero.
12. Increase the additional class count until the attendance percentage reaches 75%, assuming every future class
13. Display the additional consecutive classes required.
14. Stop.
```

Flowchart Description

```
Flowchart: Attendance Recovery Computation

START
|
v
```



Pseudocode

```
Pseudocode: Attendance Recovery Computation

BEGIN

INPUT total_classes
INPUT attended_classes

IF total_classes <= 0 THEN
PRINT "Invalid total classes"

ELSE IF attended_classes < 0 OR attended_classes > total_classes THEN
PRINT "Invalid number of classes attended"

ELSE
percentage = (attended_classes / total_classes) * 100
PRINT percentage

IF percentage >= 75 THEN
    PRINT "Attendance requirement satisfied"

ELSE
    required = 0

    WHILE ((attended_classes + required) /
          (total_classes + required)) * 100 < 75
        required = required + 1
    END WHILE

    PRINT required
END IF

END IF
```

```
END
```

Python Source Code

```
total_classes = int(input("Enter total classes conducted: "))
attended_classes = int(input("Enter classes attended: "))

if total_classes <= 0:
    print("Invalid total classes")
elif attended_classes < 0 or attended_classes > total_classes:
    print("Invalid number of classes attended")
else:
    percentage = attended_classes / total_classes * 100
    print(f"Current attendance: {percentage:.2f}%")

    if percentage >= 75:
        print("Attendance requirement satisfied")
    else:
        required = 0
        while (attended_classes + required) / (total_classes + required) * 100 < 75:
            required += 1
        print("Additional consecutive classes required:", required)
```

Test Cases

```
Test Cases: Attendance Recovery Computation

Test Case 1: Attendance below 75%
Input:
Total classes = 60
Attended classes = 40
Expected output:
Current attendance = 66.67%
Additional consecutive classes required = 20

Test Case 2: Attendance already satisfies 75%
Input:
Total classes = 40
Attended classes = 32
Expected output:
Current attendance = 80.00%
Attendance requirement satisfied

Test Case 3: Zero total classes
Input:
Total classes = 0
Attended classes = 0
Expected output:
Invalid total classes

Test Case 4: Attended classes exceed total classes
Input:
Total classes = 50
Attended classes = 55
Expected output:
Invalid number of classes attended

Test Case 5: No classes attended
Input:
Total classes = 20
Attended classes = 0
Expected output:
Current attendance = 0.00%
Additional consecutive classes required = 60
```

Conclusion

The Attendance Recovery Computation program calculates the current attendance percentage and checks whether the student meets the minimum attendance requirement of 75%. If the attendance is below the required percentage, the program calculates the minimum number of consecutive classes the student must attend to become eligible. Input validation prevents invalid attendance data. This case study demonstrates the use of variables, arithmetic operators, conditional statements, and while loops in Python.

Case Study 14: Warehouse Inventory Reconciliation

Problem Statement

A warehouse stores multiple products and records their opening stock, incoming quantities, dispatched quantities, and actual closing stock. The program calculates the expected closing stock for each product and compares it with the actual closing stock. It identifies stock discrepancies, displays the reconciliation status, and calculates the replenishment quantity required when a shortage is detected.

Objectives

1. To calculate the expected closing stock of multiple products.
2. To compare expected stock with actual closing stock.
3. To identify stock discrepancies.
4. To determine the replenishment quantity required for stock shortages.
5. To use variables, data types, operators, conditional statements, loops, lists, and tuples in Python.

Assumptions

1. The number of products is entered by the user.
2. Product names are entered as strings.
3. Stock quantities are entered as integers.
4. Opening stock, incoming quantity, dispatched quantity, and actual closing stock are expected to be non-negative.
5. Expected closing stock is calculated as opening stock + incoming quantity - dispatched quantity.
6. A negative discrepancy indicates a stock shortage.
7. Replenishment quantity is calculated for shortages only.

Algorithm

```
Algorithm

1. Start.
2. Read the number of products.
3. Create an empty list to store product details.
4. Repeat for each product:
   a. Read the product name.
   b. Read opening stock, incoming quantity, dispatched quantity, and actual closing stock.
   c. Calculate expected closing stock = opening stock + incoming quantity - dispatched quantity.
   d. Calculate discrepancy = actual closing stock - expected closing stock.
   e. If expected closing stock is negative, display invalid stock movement.
   f. Otherwise, if discrepancy is not zero, display stock discrepancy detected and calculate replenishment for s
   g. Otherwise, display stock reconciled.
   h. Store the product details in a tuple and append it to the list.
5. Display the inventory reconciliation report for all products.
6. Stop.
```

Flowchart Description

```
Flowchart Logic

Start
↓
Input number of products
↓
Repeat for each product
```

```
↓
Input product details
↓
Calculate expected closing stock
↓
Calculate discrepancy
↓
Is expected stock negative?
■■■ Yes → Display invalid stock movement
■■■ No → Is discrepancy non-zero?
■■■ Yes → Display discrepancy and shortage replenishment
■■■ No → Display stock reconciled
↓
Store product details in a list of tuples
↓
Display inventory reconciliation report
↓
Stop
```

Pseudocode

```
Pseudocode

START

INPUT number_of_products
CREATE empty list products

FOR each product:
INPUT product_name
INPUT opening_stock
INPUT incoming_quantity
INPUT dispatched_quantity
INPUT actual_closing_stock

expected_stock = opening_stock + incoming_quantity - dispatched_quantity
discrepancy = actual_closing_stock - expected_stock

IF expected_stock < 0:
    status = "Invalid stock movement"
    replenishment = 0
ELSE IF discrepancy != 0:
    status = "Stock discrepancy detected"
    replenishment = maximum of 0 and negative discrepancy
ELSE:
    status = "Stock reconciled"
    replenishment = 0

STORE product details as a tuple in products

FOR each product in products:
DISPLAY product name
DISPLAY expected stock
DISPLAY actual stock
DISPLAY discrepancy
DISPLAY status
DISPLAY replenishment quantity

STOP
```

Python Source Code

```
n = int(input("Enter number of products: "))

products = []
```

```
for i in range(n):
    print("\nProduct", i + 1)

    name = input("Enter product name: ")
    opening = int(input("Enter opening stock: "))
    incoming = int(input("Enter incoming quantity: "))
    dispatched = int(input("Enter dispatched quantity: "))
    closing = int(input("Enter actual closing stock: "))

    expected = opening + incoming - dispatched
    discrepancy = closing - expected

    if expected < 0:
        status = "Invalid stock movement"
        replenishment = 0
    elif discrepancy != 0:
        status = "Stock discrepancy detected"
        replenishment = max(0, -discrepancy)
    else:
        status = "Stock reconciled"
        replenishment = 0

    products.append(
        (name, opening, incoming, dispatched, closing,
         expected, discrepancy, status, replenishment)
    )

print("\n===== INVENTORY RECONCILIATION REPORT =====")

for product in products:
    name, opening, incoming, dispatched, closing, expected, discrepancy, status, replenishment = product

    print("\nProduct:", name)
    print("Expected closing stock:", expected)
    print("Actual closing stock:", closing)
    print("Discrepancy:", discrepancy)
    print("Status:", status)
    print("Replenishment required:", replenishment)
```

Test Cases

```
Test Cases

Test Case 1: Stock Reconciled

Input:
Product name: Pen
Opening stock: 100
Incoming quantity: 50
Dispatched quantity: 30
Actual closing stock: 120

Expected result:
Expected closing stock: 120
Discrepancy: 0
Status: Stock reconciled
Replenishment required: 0

Test Case 2: Stock Shortage

Input:
Product name: Book
Opening stock: 80
Incoming quantity: 20
Dispatched quantity: 30
Actual closing stock: 60
```

```
Expected result:
Expected closing stock: 70
Discrepancy: -10
Status: Stock discrepancy detected
Replenishment required: 10
```

Conclusion

The Warehouse Inventory Reconciliation program calculates expected closing stock and compares it with actual stock. It identifies discrepancies, reports stock status, and calculates replenishment quantities for shortages. This case study demonstrates the use of Python variables, operators, conditional statements, loops, lists, tuples, and tuple unpacking.

Case Study 17: Student Performance Trend Analysis

Problem Statement

The program analyzes the academic performance of multiple students across three tests. It accepts each student's name and test marks, calculates the average marks, and identifies the performance trend by comparing the marks obtained in the first and third tests. The trend is classified as Improving, Declining, or Stable.

Objectives

1. To collect the names and marks of multiple students.
2. To store test marks using Python lists.
3. To calculate the average marks of each student.
4. To identify whether a student's performance is improving, declining, or stable.
5. To use variables, data types, operators, conditional statements, loops, lists, tuples, and tuple unpacking.

Assumptions

1. The number of students is entered by the user.
2. Each student has marks for exactly three tests.
3. Student names are entered as strings.
4. Test marks are entered as numbers.
5. The average is calculated using all three test marks.
6. Performance trend is determined by comparing the third test mark with the first test mark.
7. Higher third-test marks indicate an improving trend, lower marks indicate a declining trend, and equal marks indicate a stable trend.

Algorithm

```
Algorithm

1. Start.
2. Input the number of students.
3. Create an empty list to store student details.
4. Repeat for each student:
   a. Input the student's name.
   b. Create an empty list for test marks.
   c. Input marks for three tests and store them in the list.
   d. Calculate the average of the three test marks.
   e. Compare the third test mark with the first test mark.
   f. If the third mark is greater, set the trend to Improving.
   g. If the third mark is lower, set the trend to Declining.
   h. Otherwise, set the trend to Stable.
   i. Store the student's details as a tuple in the list.
5. Display each student's name, test marks, average marks, and performance trend.
6. Stop.
```

Flowchart Description

```
Flowchart Logic

Start
↓
Input number of students
↓
```

```
Create empty students list
↓
Input student name and three test marks
↓
Calculate average marks
↓
Is third test mark greater than first?
■■■ Yes → Trend = Improving
■■■ No → Is third test mark lower than first?
■■■ Yes → Trend = Declining
■■■ No → Trend = Stable
↓
Store student details as a tuple
↓
Are more students remaining?
■■■ Yes → Input next student's details
■■■ No → Display student performance report
↓
Stop
```

Pseudocode

```
Pseudocode

START

INPUT number_of_students
CREATE empty list students

FOR each student:
INPUT student_name
CREATE empty list marks

FOR test_number from 1 to 3:
    INPUT test_mark
    ADD test_mark to marks

average = SUM(marks) / 3

IF marks[2] > marks[0]:
    trend = "Improving"
ELSE IF marks[2] < marks[0]:
    trend = "Declining"
ELSE:
    trend = "Stable"

STORE (student_name, marks, average, trend) in students

FOR each student in students:
DISPLAY student_name
DISPLAY marks
DISPLAY average
DISPLAY trend

STOP
```

Python Source Code

```
n = int(input("Enter number of students: "))

students = []

for i in range(n):
    print("\nStudent", i + 1)

    name = input("Enter student name: ")
```

```
        marks = []

        for j in range(3):
            mark = float(input(f"Enter marks for test {j + 1}: "))
            marks.append(mark)

        average = sum(marks) / len(marks)

        if marks[2] > marks[0]:
            trend = "Improving"
        elif marks[2] < marks[0]:
            trend = "Declining"
        else:
            trend = "Stable"

        students.append((name, marks, average, trend))

    print("\n===== STUDENT PERFORMANCE REPORT =====")

    for student in students:
        name, marks, average, trend = student

        print("\nStudent:", name)
        print("Test marks:", marks)
        print(f"Average marks: {average:.2f}")
        print("Performance trend:", trend)
```

Test Cases

```
Test Cases

Test Case 1: Improving Performance

Input:
Student name: Ravi
Test 1 marks: 40
Test 2 marks: 60
Test 3 marks: 80

Expected output:
Test marks: [40.0, 60.0, 80.0]
Average marks: 60.00
Performance trend: Improving

Test Case 2: Declining Performance

Input:
Student name: Sita
Test 1 marks: 80
Test 2 marks: 70
Test 3 marks: 60

Expected output:
Test marks: [80.0, 70.0, 60.0]
Average marks: 70.00
Performance trend: Declining

Test Case 3: Stable Performance

Input:
Student name: Arun
Test 1 marks: 75
Test 2 marks: 65
Test 3 marks: 75

Expected output:
```

```
Test marks: [75.0, 65.0, 75.0]
Average marks: 71.67
Performance trend: Stable
```

Conclusion

The Student Performance Trend Analysis program calculates the average marks of students and identifies their performance trends by comparing the first and third test marks. It classifies performance as Improving, Declining, or Stable. This case study demonstrates the use of Python variables, operators, loops, conditional statements, lists, tuples, and tuple unpacking.

Case Study 28: Repeated Transaction Detection

Problem Statement

Repeated transactions can occur when the same transaction ID appears more than once in a transaction record. This program accepts multiple transaction IDs, detects repeated IDs, and displays a report containing each repeated ID only once.

Objectives

1. To accept multiple transaction IDs from the user.
2. To identify repeated transaction IDs.
3. To avoid displaying the same repeated ID multiple times.
4. To display a transaction audit report.
5. To use Python variables, strings, lists, loops, conditional statements, and list operations.

Assumptions

1. The number of transactions is entered by the user.
2. Transaction IDs are entered as strings.
3. Transaction IDs are compared exactly as entered.
4. A transaction ID is considered repeated if it appears more than once.
5. Each repeated transaction ID is displayed only once in the report.
6. The program processes the transaction IDs entered during one execution.

Algorithm

```
Algorithm

1. Start.
2. Input the number of transactions.
3. Create two empty lists: transactions and repeated.
4. Repeat for each transaction:
   a. Input the transaction ID.
   b. Check whether the ID already exists in transactions.
   c. If it exists and is not already in repeated, add it to repeated.
   d. Otherwise, if it does not exist in transactions, add it to transactions.
5. Display the transaction audit report.
6. If repeated is empty, display that no repeated transactions were found.
7. Otherwise, display all repeated transaction IDs.
8. Stop.
```

Flowchart Description

```
Flowchart Logic

Start
↓
Input number of transactions
↓
Create empty transactions and repeated lists
↓
Input transaction ID
↓
Is ID already in transactions?
■■■ Yes → Is ID already in repeated?
■             ■■■ Yes → Continue
```

```
■             ■■■ No → Add ID to repeated
■■■ No → Add ID to transactions
↓
Are more transactions remaining?
■■■ Yes → Input next transaction ID
■■■ No → Display audit report
↓
Is repeated list empty?
■■■ Yes → Display no repeated transactions
■■■ No → Display repeated transaction IDs
↓
Stop
```

Pseudocode

```
Pseudocode

START

INPUT number_of_transactions

CREATE empty list transactions
CREATE empty list repeated

FOR each transaction:
INPUT transaction_id

IF transaction_id is in transactions:
    IF transaction_id is not in repeated:
        ADD transaction_id to repeated
ELSE:
    ADD transaction_id to transactions

DISPLAY transaction audit report

IF repeated is empty:
DISPLAY "No repeated transactions found"
ELSE:
FOR each transaction_id in repeated:
DISPLAY transaction_id

STOP
```

Python Source Code

```
n = int(input("Enter number of transactions: "))

transactions = []
repeated = []

for i in range(n):
    transaction_id = input("Enter transaction ID: ")

    if transaction_id in transactions:
        if transaction_id not in repeated:
            repeated.append(transaction_id)
    else:
        transactions.append(transaction_id)

print("\n===== TRANSACTION AUDIT REPORT =====")

if len(repeated) == 0:
    print("No repeated transactions found.")
else:
    print("Repeated transaction IDs:")
    for transaction_id in repeated:
```

```
        print(transaction_id)
```

Test Cases

```
Test Cases

Test Case 1: Repeated Transaction IDs

Input:
Number of transactions: 6
Transaction IDs: TX101, TX102, TX103, TX101, TX104, TX102

Expected output:
Repeated transaction IDs:
TX101
TX102

Test Case 2: No Repeated Transactions

Input:
Number of transactions: 3
Transaction IDs: TX201, TX202, TX203

Expected output:
No repeated transactions found.
```

Conclusion

The Repeated Transaction Detection program identifies transaction IDs that appear more than once and displays each repeated ID only once. It demonstrates the use of Python strings, lists, loops, conditional statements, membership operators, and list operations.

Case Study 31: String-Based Examination Answer Evaluation

Problem Statement

To develop a Python program that evaluates a student's examination answers by comparing them with predefined correct answers. The program displays whether each answer is correct or incorrect and calculates the student's final score.

Objectives

1. To accept answers from a student.
2. To compare student answers with the correct answers.
3. To identify correct and incorrect answers.
4. To calculate the total score.
5. To display the examination result.
6. To practice strings, lists, loops, conditional statements, and string operations in Python.

Assumptions

1. The examination contains three predefined questions.
2. Each question has one predefined correct answer.
3. Answers are evaluated using text comparison.
4. Uppercase and lowercase differences are ignored.
5. Leading and trailing spaces in student answers are removed.
6. Each correct answer receives one mark.
7. No negative marks are awarded.

Algorithm

```
Step 1: Start.
Step 2: Store the questions and correct answers in lists.
Step 3: Create an empty list to store student answers.
Step 4: Initialize the score to zero.
Step 5: Display each question and accept the student's answer.
Step 6: Remove leading and trailing spaces and convert the answer to lowercase.
Step 7: Store the student's answer.
Step 8: Compare the student's answer with the correct answer.
Step 9: If the answer matches, increase the score by one.
Step 10: Repeat Steps 5 to 9 for all questions.
Step 11: Display each question, student answer, correct answer, and result.
Step 12: Display the number of correct and incorrect answers and the final score.
Step 13: Stop.
```

Flowchart Description

```
Start
  |
Store questions and correct answers
  |
Initialize score and student answer list
  |
Display question and accept answer
  |
Normalize and store student answer
  |
Compare answer with correct answer
```

```
  |
Is the answer correct?
  | Yes: Increase score
  | No: Keep score unchanged
  |
Are more questions available?
  | Yes: Repeat question and answer steps
  | No
  |
Display results and final score
  |
Stop
```

Pseudocode

```
BEGIN
    Store questions and correct answers
    Set score = 0
    Create empty student_answers list

    FOR each question
        Display question
        Input student answer
        Remove extra spaces
        Convert answer to lowercase
        Store student answer

        IF student answer equals correct answer
            Increase score by 1
        END IF
    END FOR

    Display all answers and their results
    Display correct answers count
    Display incorrect answers count
    Display final score
END
```

Python Source Code

```
# Case Study 31: String-Based Examination Answer Evaluation

questions = [
    "What is the capital of India?",
    "Which language are we learning?",
    "What does CPU stand for?"
]

correct_answers = [
    "New Delhi",
    "Python",
    "Central Processing Unit"
]

n = len(questions)
student_answers = []
score = 0

print("===== EXAMINATION ANSWER EVALUATION =====")

for i in range(n):
    print("\nQuestion", i + 1, ":", questions[i])
    answer = input("Enter your answer: ").strip().lower()
    student_answers.append(answer)

    if answer == correct_answers[i].strip().lower():
```

```
            score += 1

    print("\n===== RESULT =====")

    for i in range(n):
        print("Question", i + 1, ":", questions[i])
        print("Your answer:", student_answers[i])
        print("Correct answer:", correct_answers[i])

        if student_answers[i] == correct_answers[i].strip().lower():
            print("Result: Correct")
        else:
            print("Result: Incorrect")

    print("\nTotal questions:", n)
    print("Correct answers:", score)
    print("Incorrect answers:", n - score)
    print("Final score:", score, "/", n)
```

Test Cases

```
Test Case 1: All answers correct
Input:
New Delhi
Python
Central Processing Unit
Expected Output:
Correct answers: 3
Incorrect answers: 0
Final score: 3 / 3

Test Case 2: One answer incorrect
Input:
Mumbai
Python
Central Processing Unit
Expected Output:
Correct answers: 2
Incorrect answers: 1
Final score: 2 / 3

Test Case 3: All answers incorrect
Input:
Mumbai
Java
Computer Processing Unit
Expected Output:
Correct answers: 0
Incorrect answers: 3
Final score: 0 / 3
```

Conclusion

The String-Based Examination Answer Evaluation program successfully compares student answers with predefined correct answers. It identifies correct and incorrect responses and calculates the final score. The program demonstrates the use of strings, lists, loops, conditional statements, and string operations in Python.

Case Study 35: Text Compression Using Character Runs

Problem Statement

To develop a Python program that compresses a string by counting consecutive occurrences of the same character. The program represents each consecutive character run using the character followed by its count.

Objectives

1. To accept a string from the user.
2. To identify consecutive repeated characters.
3. To count the occurrences of each consecutive character.
4. To generate a compressed representation of the string.
5. To display the original and compressed strings and their lengths.
6. To practice strings, loops, conditional statements, and string operations in Python.

Assumptions

1. The input is treated as a string.
2. Characters are compared exactly, including uppercase and lowercase letters.
3. Only consecutive occurrences of the same character are grouped together.
4. Each run is represented by the character followed by its count.
5. An empty string produces an empty compressed string.
6. The compressed representation may be longer than the original string.

Algorithm

```
Step 1: Start.
Step 2: Accept a string from the user.
Step 3: Initialize the compressed string as empty.
Step 4: If the input is not empty, initialize the character count to one.
Step 5: Traverse the string from the second character.
Step 6: If the current character matches the previous character, increase the count.
Step 7: Otherwise, append the previous character and its count to the compressed string, then reset the count to
Step 8: After the loop, append the final character and its count.
Step 9: Display the original string and compressed string.
Step 10: Display the lengths of both strings.
Step 11: Stop.
```

Flowchart Description

```
Start
  |
Input string
  |
Initialize compressed string
  |
Is the string empty?
  | Yes: Display empty compressed string
  | No
  |
Initialize count = 1
  |
Compare current character with previous character
  |
Are they equal?
  | Yes: Increase count
```

```
  | No: Append previous character and count; reset count
  |
Are more characters available?
  | Yes: Repeat comparison
  | No: Append final character and count
  |
Display original and compressed strings and their lengths
  |
Stop
```

Pseudocode

```
BEGIN
    INPUT text
    compressed = ""

    IF length of text > 0 THEN
        count = 1

        FOR i from 1 to length of text - 1
            IF text[i] equals text[i - 1] THEN
                count = count + 1
            ELSE
                Append text[i - 1] and count to compressed
                count = 1
            END IF
        END FOR

        Append last character and count to compressed
    END IF

    DISPLAY original text
    DISPLAY compressed text
    DISPLAY original length
    DISPLAY compressed length
END
```

Python Source Code

```
# Case Study 35: Text Compression Using Character Runs

text = input("Enter a string to compress: ")

compressed = ""

if len(text) > 0:
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            compressed += text[i - 1] + str(count)
            count = 1

    compressed += text[-1] + str(count)

print("\n===== TEXT COMPRESSION REPORT =====")
print("Original text:", text)
print("Compressed text:", compressed)
print("Original length:", len(text))
print("Compressed length:", len(compressed))
```

Test Cases

```
Test Case 1:
```

```
Input: aaabbc
Expected compressed output: a3b2c1

Test Case 2:
Input: aaaa
Expected compressed output: a4

Test Case 3:
Input: abc
Expected compressed output: a1b1c1

Test Case 4:
Input: aabbaa
Expected compressed output: a2b2a2

Test Case 5:
Input: empty string
Expected compressed output: empty string
```

Conclusion

The Text Compression Using Character Runs program successfully groups consecutive identical characters and represents each group using the character and its count. It displays the original string, compressed string, and their lengths. This program demonstrates the use of strings, loops, conditional statements, and string operations in Python.

Case Study 42: Student Marks Matrix Analysis

Problem Statement

To develop a Python program that stores student marks in a two-dimensional list (matrix). The program calculates and displays each student's total and average marks, along with the average marks for each subject.

Objectives

1. To accept marks for multiple students and subjects.
2. To store marks in a two-dimensional list.
3. To calculate each student's total marks.
4. To calculate each student's average marks.
5. To calculate the average marks for each subject.
6. To practice nested lists, nested loops, variables, arithmetic operators, and formatted output in Python.

Assumptions

1. The number of students and subjects must be positive integers.
2. Marks are entered as numbers.
3. Each student has marks for the same number of subjects.
4. The program calculates averages using the number of subjects or students entered.
5. The program does not validate whether marks fall within a particular examination range.

Algorithm

```
Step 1: Start.
Step 2: Input the number of students and subjects.
Step 3: If either value is not positive, display an invalid input message.
Step 4: Otherwise, create an empty marks matrix.
Step 5: Input each student's marks and store them in a row of the matrix.
Step 6: Display the marks matrix.
Step 7: For each student, calculate and display total marks and average marks.
Step 8: For each subject, add the marks of all students.
Step 9: Calculate and display the average for each subject.
Step 10: Stop.
```

Flowchart Description

```
Start
  |
Input number of students and subjects
  |
Are both values positive?
  | No: Display invalid input message and Stop
  | Yes
  |
Create marks matrix
  |
Input and store marks using nested loops
  |
Display student marks
  |
Calculate and display each student's total and average
  |
Calculate and display each subject's average
  |
```

```
Stop
```

Pseudocode

```
BEGIN
    INPUT number_of_students
    INPUT number_of_subjects

    IF either number is less than or equal to zero
        DISPLAY invalid input message
    ELSE
        CREATE empty marks_matrix

        FOR each student
            CREATE empty student_marks
            FOR each subject
                INPUT mark
                ADD mark to student_marks
            END FOR
            ADD student_marks to marks_matrix
        END FOR

        FOR each student
            CALCULATE total marks
            CALCULATE average marks
            DISPLAY marks, total, and average
        END FOR

        FOR each subject
            CALCULATE total marks for that subject
            CALCULATE subject average
            DISPLAY subject average
        END FOR
    END IF
END
```

Python Source Code

```
# Case Study 42: Student Marks Matrix Analysis

rows = int(input("Enter number of students: "))
cols = int(input("Enter number of subjects: "))

if rows <= 0 or cols <= 0:
    print("Invalid number of students or subjects")
else:
    marks_matrix = []

    for i in range(rows):
        student_marks = []
        print("\nStudent", i + 1)

        for j in range(cols):
            mark = float(input(f"Enter marks for subject {j + 1}: "))
            student_marks.append(mark)

        marks_matrix.append(student_marks)

    print("\n===== STUDENT MARKS MATRIX =====")

    for i in range(rows):
        print("Student", i + 1, ":", marks_matrix[i])
        total = sum(marks_matrix[i])
        average = total / cols
        print("Total marks:", total)
        print(f"Average marks: {average:.2f}")
```

```
    print("\n===== SUBJECT-WISE AVERAGES =====")

    for j in range(cols):
        subject_total = 0

        for i in range(rows):
            subject_total += marks_matrix[i][j]

        subject_average = subject_total / rows
        print(f"Subject {j + 1} average: {subject_average:.2f}")
```

Test Cases

```
Test Case 1: Two students and three subjects

Input:
Student 1: 80, 90, 70
Student 2: 60, 75, 85

Expected output:
Student 1 total: 240
Student 1 average: 80.00
Student 2 total: 220
Student 2 average: 73.33
Subject 1 average: 70.00
Subject 2 average: 82.50
Subject 3 average: 77.50

Test Case 2: Invalid number of students

Input:
Number of students: 0
Number of subjects: 3

Expected output:
Invalid number of students or subjects
```

Conclusion

The Student Marks Matrix Analysis program successfully stores marks in a two-dimensional list and calculates student-wise totals and averages, as well as subject-wise averages. It demonstrates the use of nested lists, nested loops, arithmetic operations, and conditional statements in Python.

Case Study 49: Tuple-Based Product Catalogue Audit

Problem Statement

To develop a Python program that audits a product catalogue stored using tuples. The program identifies duplicate product IDs, detects products with stock quantities below five, and displays the complete product catalogue.

Objectives

1. To accept product details from the user.
2. To store each product as a tuple.
3. To identify duplicate product IDs.
4. To identify products with stock quantities below five.
5. To display the complete product catalogue.
6. To practice tuples, lists, tuple unpacking, loops, conditional statements, and membership operators in Python.

Assumptions

1. The number of product records is entered as an integer.
2. Product IDs and names are entered as strings.
3. Product prices are entered as numbers.
4. Stock quantities are entered as integers.
5. Product IDs are compared exactly and are case-sensitive.
6. A product is considered low-stock when its stock quantity is less than five.
7. Each duplicated product ID is reported only once.
8. The program does not validate negative prices or stock quantities.

Algorithm

```
Step 1: Start.
Step 2: Input the number of products.
Step 3: Create an empty products list.
Step 4: For each product, input its ID, name, price, and stock quantity.
Step 5: Store the product details as a tuple in the products list.
Step 6: Display the product catalogue audit heading.
Step 7: If the products list is empty, display a message.
Step 8: Otherwise, create lists for seen IDs, duplicate IDs, and low-stock products.
Step 9: Traverse each product tuple and unpack its values.
Step 10: Check whether the product ID has appeared before. If it has, add it to the duplicate list if not already
Step 11: If the stock quantity is less than five, add the product details to the low-stock list.
Step 12: Display the total number of product records.
Step 13: Display duplicate product IDs and low-stock products, or suitable messages if none exist.
Step 14: Display the complete product catalogue.
Step 15: Stop.
```

Flowchart Description

```
Start
  |
Input number of products
  |
Create products list
  |
```

```
Input product details and store as tuples
  |
Is products list empty?
  | Yes: Display no products available
  | No
  |
Initialize seen IDs, duplicate IDs, and low-stock lists
  |
Process each product tuple
  |
Check for duplicate product ID
  |
Check whether stock is less than five
  |
Are more products available?
  | Yes: Process next product
  | No
  |
Display audit results and complete catalogue
  |
Stop
```

Pseudocode

```
BEGIN
    INPUT n
    CREATE empty products list

    FOR each product from 1 to n
        INPUT product_id, name, price, stock
        STORE (product_id, name, price, stock) in products
    END FOR

    IF products is empty
        DISPLAY no products available
    ELSE
        CREATE empty seen_ids, duplicate_ids, low_stock lists

        FOR each product in products
            UNPACK product details

            IF product_id is in seen_ids
                IF product_id is not in duplicate_ids
                    ADD product_id to duplicate_ids
                END IF
            ELSE
                ADD product_id to seen_ids
            END IF

            IF stock is less than 5
                ADD product details to low_stock
            END IF
        END FOR

        DISPLAY total product records
        DISPLAY duplicate product IDs or None
        DISPLAY low-stock products or None
        DISPLAY complete catalogue
    END IF
END
```

Python Source Code

```
# Case Study 49: Tuple-Based Product Catalogue Audit

n = int(input("Enter number of products: "))
```

```
products = []

for i in range(n):
    print("\nProduct", i + 1)

    product_id = input("Enter product ID: ")
    name = input("Enter product name: ")
    price = float(input("Enter product price: "))
    stock = int(input("Enter stock quantity: "))

    products.append((product_id, name, price, stock))

print("\n===== PRODUCT CATALOGUE AUDIT =====")

if len(products) == 0:
    print("No products available for audit.")
else:
    seen_ids = []
    duplicate_ids = []
    low_stock = []

    for product in products:
        product_id, name, price, stock = product

        if product_id in seen_ids:
            if product_id not in duplicate_ids:
                duplicate_ids.append(product_id)
        else:
            seen_ids.append(product_id)

        if stock < 5:
            low_stock.append((product_id, name, stock))

    print("\nTotal product records:", len(products))

    if len(duplicate_ids) == 0:
        print("Duplicate product IDs: None")
    else:
        print("Duplicate product IDs:", duplicate_ids)

    if len(low_stock) == 0:
        print("Low-stock products: None")
    else:
        print("\nLow-stock products:")
        for item in low_stock:
            product_id, name, stock = item
            print(product_id, "-", name, "- Stock:", stock)

    print("\nComplete catalogue:")
    for product in products:
        product_id, name, price, stock = product
        print(product_id, name, "Price:", price, "Stock:", stock)
```

Test Cases

```
Test Case 1: Duplicate ID and low stock

Input:
Product 1: P101, Pen, 10, 20
Product 2: P102, Book, 50, 3
Product 3: P102, Notebook, 40, 8

Expected results:
Total product records: 3
Duplicate product IDs: ['P102']
Low-stock products: Book, stock 3
```

```
Test Case 2: No duplicate IDs and no low stock

Input:
Product 1: P201, Pencil, 5, 10
Product 2: P202, Bag, 200, 8

Expected results:
Duplicate product IDs: None
Low-stock products: None

Test Case 3: No products

Input:
Number of products: 0

Expected result:
No products available for audit.
```

Conclusion

The Tuple-Based Product Catalogue Audit program successfully stores product records as tuples and identifies duplicate product IDs and products with stock quantities below five. It displays the audit findings and the complete catalogue. The program demonstrates tuples, lists, tuple unpacking, loops, conditional statements, and membership operations in Python.

Case Study 55: Tuple-Based Transaction Audit

Problem Statement

To develop a Python program that audits financial transactions stored as tuples. The program calculates total credits, total debits, and net balance change, identifies duplicate transaction IDs, and displays the complete transaction list.

Objectives

1. To accept transaction details from the user.
2. To store each transaction as a tuple.
3. To calculate total credit amounts.
4. To calculate total debit amounts.
5. To calculate the net balance change.
6. To identify duplicate transaction IDs.
7. To practice tuples, lists, tuple unpacking, loops, conditional statements, and arithmetic operations in Python.

Assumptions

1. The number of transactions is entered as an integer.
2. Transaction IDs are entered as strings.
3. Transaction amounts are entered as numbers.
4. Transaction types are entered as Credit or Debit.
5. Transaction types are converted to lowercase before checking.
6. Transaction IDs are compared exactly and are case-sensitive.
7. Each duplicate transaction ID is reported only once.
8. Amounts are assumed to be non-negative.
9. Only valid Credit and Debit transaction types are expected.

Algorithm

```
Step 1: Start.
Step 2: Input the number of transactions.
Step 3: Create an empty transactions list.
Step 4: Input the ID, amount, and type for each transaction.
Step 5: Store each transaction as a tuple.
Step 6: If there are no transactions, display a suitable message.
Step 7: Otherwise, initialize credit and debit totals to zero and create lists for seen IDs and duplicate IDs.
Step 8: Traverse each transaction tuple and unpack its values.
Step 9: Check whether the transaction ID has appeared before and record duplicates without repeating them.
Step 10: If the transaction type is credit, add its amount to total credits.
Step 11: If the transaction type is debit, add its amount to total debits.
Step 12: Calculate net balance change by subtracting total debits from total credits.
Step 13: Display totals, duplicate IDs, and the complete transaction list.
Step 14: Stop.
```

Flowchart Description

```
Start
  |
Input number of transactions
  |
```

```
Input transaction details and store as tuples
  |
Are transactions available?
  | No: Display no transactions available
  | Yes
  |
Initialize credit and debit totals and audit lists
  |
Process each transaction tuple
  |
Check for duplicate transaction ID
  |
Is transaction type Credit?
  | Yes: Add amount to total credits
  | No
  |
Is transaction type Debit?
  | Yes: Add amount to total debits
  | No
  |
Are more transactions available?
  | Yes: Process next transaction
  | No
  |
Calculate net balance change
  |
Display audit report and transaction list
  |
Stop
```

Pseudocode

```
BEGIN
    INPUT n
    CREATE empty transactions list

    FOR each transaction from 1 to n
        INPUT transaction_id, amount, transaction_type
        CONVERT transaction_type to lowercase
        STORE transaction as a tuple
    END FOR

    IF transactions is empty
        DISPLAY no transactions available
    ELSE
        SET total_credit = 0
        SET total_debit = 0
        CREATE empty seen_ids and duplicate_ids lists

        FOR each transaction
            UNPACK transaction details

            IF transaction_id is in seen_ids
                IF transaction_id is not in duplicate_ids
                    ADD transaction_id to duplicate_ids
                END IF
            ELSE
                ADD transaction_id to seen_ids
            END IF

            IF transaction_type equals credit
                ADD amount to total_credit
            ELSE IF transaction_type equals debit
                ADD amount to total_debit
            END IF
        END FOR
```

```
        CALCULATE net balance change
        DISPLAY totals and duplicate IDs
        DISPLAY complete transaction list
    END IF
END
```

Python Source Code

```
# Case Study 55: Tuple-Based Transaction Audit

n = int(input("Enter number of transactions: "))

transactions = []

for i in range(n):
    print("\nTransaction", i + 1)

    transaction_id = input("Enter transaction ID: ")
    amount = float(input("Enter transaction amount: "))
    transaction_type = input("Enter type (Credit/Debit): ").strip().lower()

    transactions.append((transaction_id, amount, transaction_type))

print("\n===== TRANSACTION AUDIT REPORT =====")

if len(transactions) == 0:
    print("No transactions available for audit.")
else:
    total_credit = 0
    total_debit = 0
    seen_ids = []
    duplicate_ids = []

    for transaction in transactions:
        transaction_id, amount, transaction_type = transaction

        if transaction_id in seen_ids:
            if transaction_id not in duplicate_ids:
                duplicate_ids.append(transaction_id)
        else:
            seen_ids.append(transaction_id)

        if transaction_type == "credit":
            total_credit += amount
        elif transaction_type == "debit":
            total_debit += amount

    print("Total transactions:", len(transactions))
    print(f"Total credits: {total_credit:.2f}")
    print(f"Total debits: {total_debit:.2f}")
    print(f"Net balance change: {total_credit - total_debit:.2f}")

    if len(duplicate_ids) == 0:
        print("Duplicate transaction IDs: None")
    else:
        print("Duplicate transaction IDs:", duplicate_ids)

    print("\nComplete transaction list:")
    for transaction in transactions:
        transaction_id, amount, transaction_type = transaction
        print(transaction_id, amount, transaction_type.title())
```

Test Cases

```
Test Case 1: Credits, debits, and duplicate ID
```

```
Input:
T101, 1000, Credit
T102, 300, Debit
T101, 500, Credit

Expected results:
Total transactions: 3
Total credits: 1500.00
Total debits: 300.00
Net balance change: 1200.00
Duplicate transaction IDs: ['T101']

Test Case 2: No duplicate IDs

Input:
T201, 200, Credit
T202, 50, Debit

Expected results:
Total credits: 200.00
Total debits: 50.00
Net balance change: 150.00
Duplicate transaction IDs: None

Test Case 3: No transactions

Input:
Number of transactions: 0

Expected result:
No transactions available for audit.
```

Conclusion

The Tuple-Based Transaction Audit program successfully stores transaction records as tuples, calculates total credits and debits, determines the net balance change, and identifies duplicate transaction IDs. It demonstrates the use of tuples, lists, tuple unpacking, loops, conditional statements, and arithmetic operations in Python.

Case Study 66: Student Academic Risk Classification

Problem Statement

Develop a Python program to classify students into High Risk, Moderate Risk, or Low Risk based on their attendance percentage and average marks. The program should accept details for multiple students and display their academic risk category.

Objectives

1. To accept student names, attendance percentages, and average marks.
2. To classify students into three academic risk categories.
3. To use conditional statements to determine academic risk.
4. To store student details using tuples inside a list.
5. To display an academic risk report for all students.

Assumptions

1. The number of students is entered by the user.
2. Student names are entered as strings.
3. Attendance percentages and average marks are entered as numeric values.
4. High Risk: attendance is below 75% OR average marks are below 40.
5. Moderate Risk: if not High Risk, attendance is below 85% OR average marks are below 60.
6. Otherwise, the student is classified as Low Risk.
7. Inputs are assumed to be valid; the program does not validate the entered ranges.

Algorithm

```
1. Start.
2. Input the number of students.
3. Create an empty list named students.
4. Repeat for each student:
   a. Input the student's name.
   b. Input attendance percentage.
   c. Input average marks.
   d. If attendance is below 75 or average marks are below 40, assign High Risk.
   e. Otherwise, if attendance is below 85 or average marks are below 60, assign Moderate Risk.
   f. Otherwise, assign Low Risk.
   g. Store the details and risk category as a tuple in the students list.
5. Display the academic risk report.
6. Stop.
```

Flowchart Description

```
Start
↓
Input number of students
↓
Create an empty students list
↓
Input student name, attendance, and average marks
↓
Is attendance < 75 OR average marks < 40?
■■■ Yes → Assign High Risk
■■■ No → Is attendance < 85 OR average marks < 60?
■■■ Yes → Assign Moderate Risk
■■■ No → Assign Low Risk
```

```
↓
Store student details as a tuple
↓
Are more students remaining?
■■■ Yes → Input the next student's details
■■■ No → Display the academic risk report
↓
Stop
```

Pseudocode

```
BEGIN
INPUT n
CREATE empty list students

FOR each student from 1 to n
    INPUT name
    INPUT attendance
    INPUT average_marks

    IF attendance < 75 OR average_marks < 40 THEN
        risk = "High Risk"
    ELSE IF attendance < 85 OR average_marks < 60 THEN
        risk = "Moderate Risk"
    ELSE
        risk = "Low Risk"
    END IF

    STORE (name, attendance, average_marks, risk) in students
END FOR

FOR each student in students
    DISPLAY student details and risk
END FOR

END
```

Python Source Code

```
# Case Study 66: Student Academic Risk Classification

n = int(input("Enter number of students: "))

students = []

for i in range(n):
    print("\nStudent", i + 1)

    name = input("Enter student name: ")
    attendance = float(input("Enter attendance percentage: "))
    average_marks = float(input("Enter average marks: "))

    if attendance < 75 or average_marks < 40:
        risk = "High Risk"
    elif attendance < 85 or average_marks < 60:
        risk = "Moderate Risk"
    else:
        risk = "Low Risk"

    students.append((name, attendance, average_marks, risk))

print("\n===== STUDENT ACADEMIC RISK REPORT =====")

for student in students:
    name, attendance, average_marks, risk = student
```

```
    print("\nStudent:", name)
    print("Attendance:", attendance, "%")
    print("Average marks:", average_marks)
    print("Academic risk:", risk)
```

Test Cases

```
Test Case 1:
Input:
Student name: Ravi
Attendance: 60
Average marks: 35
Expected output: High Risk

Test Case 2:
Input:
Student name: Sita
Attendance: 80
Average marks: 55
Expected output: Moderate Risk

Test Case 3:
Input:
Student name: Arun
Attendance: 90
Average marks: 75
Expected output: Low Risk

Result:
All three test cases produce the expected risk categories.
```

Conclusion

The Student Academic Risk Classification program classifies students according to their attendance percentages and average marks. It demonstrates the use of variables, data types, lists, tuples, loops, tuple unpacking, and conditional statements in Python.